

RESUMEN – CLASES DE FLUJOS DE BYTES

Clase	Dirección	Uso principal
<code>InputStream</code>	Lectura	Clase abstracta base para leer bytes
<code>OutputStream</code>	Escritura	Clase abstracta base para escribir bytes
<code>FileInputStream</code>	Lectura	Leer bytes desde un archivo
<code>FileOutputStream</code>	Escritura	Escribir bytes en un archivo

MÉTODOS CLAVE

<code>fis.read()</code> lee 1 byte → int 0-255 o -1 (fin)	<code>fos.write(b)</code> escribe 1 byte en el archivo
<code>fos.write(byte[])</code> escribe array de bytes	<code>str.getBytes()</code> convierte String → byte[]
<code>fis.available()</code> bytes disponibles sin bloquear	<code>fis.skip(n)</code> salta n bytes hacia adelante
<code>stream.close()</code> cierra el flujo (siempre)	

⚡ SOBRESCRIBIR VS AÑADIR

```
// Sobrescribe (por defecto)
FileOutputStream fos = new FileOutputStream("f.txt");

// Añade al final sin borrar
FileOutputStream fos = new FileOutputStream("f.txt", true);
```

El parámetro `true` activa modo *append*. Sin él, el archivo se vacía al abrirlo aunque tenga contenido.

FUENTES COMPATIBLES CON FLUJOS

Fuente / Destino	Clase
Archivo en disco	<code>FileInputStream</code> / <code>FileOutputStream</code>
Teclado (entrada estándar)	<code>System.in</code> (<code>InputStream</code>)
Pantalla (salida estándar)	<code>System.out</code> (<code>PrintStream</code>)
Array en memoria	<code>ByteArrayInputStream</code>
Conexión de red	<code>Socket.getInputStream()</code>

✗ ERRORES COMUNES

- No cerrar el flujo → datos sin escribir al disco, archivo bloqueado, *resource leak*
- No capturar `IOException` → error de compilación (excepción comprobada)
- Cast `(char)` antes de comparar con `-1` → bucle infinito: `(char)-1 = '\uFFFF' ≠ -1`
- Ruta incorrecta → `FileNotFoundException` (subclase de `IOException`)
- Leer byte a byte en archivos grandes es lento → usar `BufferedInputStream` (Bloque 2)

✅ TRY-WITH-RESOURCES – FORMA MODERNA

Java 7+. El flujo se cierra automáticamente al salir del bloque `try`, aunque haya excepción. No hace falta llamar a `close()`.

UN FLUJO

```
try (FileInputStream fis =
    new FileInputStream("archivo.txt")) {
    int dato;
    while ((dato = fis.read()) ≠ -1)
        System.out.print((char) dato);
} catch (IOException e) {
    System.out.println("Error");
}
// ✓ fis.close() se llama automáticamente
```

DOS FLUJOS A LA VEZ

```
try (FileInputStream fis =
    new FileInputStream("origen.txt");
    FileOutputStream fos =
    new FileOutputStream("destino.txt")) {
    int dato;
    while ((dato = fis.read()) ≠ -1)
        fos.write(dato);
} catch (IOException e) { /* manejo */ }
```

🔍 VALOR DE RETORNO DE read()

Valor	Significado
0 - 255	Byte leído (valor numérico del byte)
-1	Fin de archivo — no hay más datos

```
int dato;
// ✓ Correcto: guardar en int, comparar, luego cast
while ((dato = fis.read()) ≠ -1) {
    System.out.print((char) dato); // cast solo aquí
}
// ✗ Incorrecto: cast antes de comparar
while ((char)(dato = fis.read()) ≠ -1) { // ¡bucle infinito!
```

🏠 PROBLEMA: LECTURA BYTE A BYTE

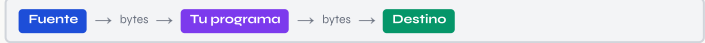
- Cada `read()` implica una llamada al sistema operativo. En archivos grandes, millones de llamadas individuales son **muy lentas**.
- Solución (Bloque 2):** `BufferedInputStream` / `BufferedOutputStream` acumulan datos en un buffer de 8 KB y reducen llamadas al SO.
- ```
// Vista previa Bloque 2
BufferedInputStream bis = new BufferedInputStream(
 new FileInputStream("archivo.txt"));
```

DAM · PROGRAMACIÓN · FLUJOS (I/O)

Flujos de Datos

BLOQUE 1 · FUNDAMENTOS · CHULETARIO · TRÍPTICO A4

CONCEPTO DE FLUJO (STREAM)



ALGORITMO UNIVERSAL DE FLUJOS

- 1 **Abrir** — `new FileInputStream("ruta")`
- 2 **Procesar** — leer con `read()` o escribir con `write()`
- 3 **Cerrar** — `close()` o try-with-resources

JERARQUÍA DE CLASES

```
java.io
├─ InputStream + abstracta, leer bytes
│ └─ FileInputStream + leer desde archivo
│ └─ OutputStream + abstracta, escribir bytes
│ └─ FileOutputStream + escribir en archivo

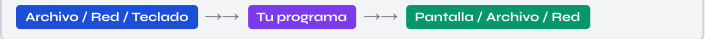
// Bloque 2 (próximo):
BufferedInputStream + lectura con buffer
BufferedOutputStream + escritura con buffer
```

CONTENIDOS DE ESTE TRÍPTICO

- Interior izq.** — Concepto de flujo, `FileInputStream`, desglose línea a línea
- Interior cent.** — `FileOutputStream`, modo *append*, copiar archivo completo
- Interior der.** — Excepciones, patrones de uso, tabla resumen del bloque
- Solapa** — Try-with-resources, valor de `read()`, problema byte a byte
- Contraportada** — Tabla de clases, métodos, fuentes, errores comunes

## 🔥 ¿QUÉ ES UN FLUJO (STREAM)?

Un **flujo** es el mecanismo de Java para leer o escribir datos. Funciona como una tubería: la información entra por un extremo, tu programa la procesa, y sale por el otro.



### FLUJOS DE BYTES – CARACTERÍSTICAS

- Trabajan con datos **binarios** (o y 1)
- `InputStream` → leer · `OutputStream` → escribir
- Son **clases abstractas** → no se instancian directamente
- Se usan sus subclases concretas: `FileInputStream`, etc.

## 📄 FILEINPUTSTREAM – LEER UN ARCHIVO

Lee un archivo **byte a byte**. Cada `read()` devuelve el siguiente byte como `int` (0-255) o `-1` al llegar al final.

### EJEMPLO COMPLETO

```
import java.io.FileInputStream;
import java.io.IOException;

public class LeerArchivo {
 public static void main(String[] args) {
 try {
 FileInputStream fis =
 new FileInputStream("archivo.txt");
 int dato;
 while ((dato = fis.read()) != -1) {
 System.out.print((char) dato);
 }
 fis.close();
 } catch (IOException e) {
 System.out.println("Error al leer");
 }
 }
}
```

### DESGLOSE LÍNEA A LÍNEA

| Código                                | Qué hace                           |
|---------------------------------------|------------------------------------|
| <code>new FileInputStream("f")</code> | Abre el archivo para lectura       |
| <code>int dato</code>                 | Variable int (0-255 o -1)          |
| <code>fis.read()</code>               | Lee 1 byte, avanza el cursor       |
| <code>!= -1</code>                    | Condición de fin de archivo        |
| <code>(char) dato</code>              | Convierte número a carácter ASCII  |
| <code>fis.close()</code>              | Cierra el flujo, libera el recurso |
| <code>catch IOException</code>        | Obligatorio – excepción comprobada |

## ⚙️ ALGORITMO UNIVERSAL – LOS 3 PASOS

- 1 Abrir** – `new FileInputStream("ruta")`  
Puede lanzar `FileNotFoundException` si no existe
- 2 Procesar** – bucle `while (read() != -1)`  
Puede lanzar `IOException` durante la lectura
- 3 Cerrar** – `close()` o try-with-resources  
Siempre obligatorio, incluso si hay error

Si no cierras el flujo → datos sin escribir, archivo bloqueado, *resource leak*.

## 🔥 FILEOUTPUTSTREAM – ESCRIBIR EN UN ARCHIVO

Escribe datos en un archivo. Por defecto **sobrescribe** el contenido existente. El parámetro `true` activa el modo `append`.

### EJEMPLO COMPLETO

```
import java.io.FileOutputStream;
import java.io.IOException;

public class EscribirArchivo {
 public static void main(String[] args) {
 try {
 FileOutputStream fos =
 new FileOutputStream("archivo.txt");
 String texto = "Hola alumnos";
 fos.write(texto.getBytes());
 fos.close();
 } catch (IOException e) {
 System.out.println("Error al escribir");
 }
 }
}
```

| Código                                 | Qué hace                                         |
|----------------------------------------|--------------------------------------------------|
| <code>new FileOutputStream("f")</code> | Crea/abre para escritura, borra contenido previo |
| <code>texto.getBytes()</code>          | Convierte String en array de bytes               |
| <code>fos.write(byte[])</code>         | Escribe todos los bytes de una vez               |

### MODO APPEND – AÑADIR SIN BORRAR

```
FileOutputStream fos =
 new FileOutputStream("archivo.txt", true);
```

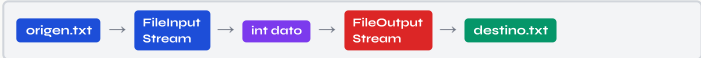
## 📄 EJEMPLO COMPLETO – COPIAR UN ARCHIVO

Patrón fundamental: combina ambos flujos para copiar cualquier archivo (texto o binario) sin pérdida de datos.

```
import java.io.*;

public class CopiarArchivo {
 public static void main(String[] args) {
 try {
 FileInputStream fis =
 new FileInputStream("origen.txt");
 FileOutputStream fos =
 new FileOutputStream("destino.txt")
 } {
 int dato;
 while ((dato = fis.read()) != -1) {
 fos.write(dato);
 }
 System.out.println("Copia completada");
 } catch (IOException e) {
 System.out.println("Error en la copia");
 }
 }
}
```

### FLUJO DE DATOS AL COPIAR



## ⚠️ MANEJO DE EXCEPCIONES EN FLUJOS

| Excepción                          | Cuándo se lanza                                                               |
|------------------------------------|-------------------------------------------------------------------------------|
| <code>FileNotFoundException</code> | Archivo inexistente o ruta incorrecta. Subclase de <code>IOException</code> . |
| <code>IOException</code>           | Error de E/S durante lectura/escritura (disco lleno, permisos...)             |
| <code>SecurityException</code>     | Sin permisos de acceso al archivo (no comprobada)                             |

Son **excepciones comprobadas** (checked): el compilador obliga a capturarlas con `catch` o declararlas con `throws`.

```
// Opción 1: capturar (recomendado)
try { ... } catch (IOException e) { ... }

// Opción 2: propagar
public static void main(String[] args) throws IOException { ... }
```

## 🔗 PATRONES DE USO FRECUENTES

### LEER TODO EL CONTENIDO COMO STRING

```
StringBuilder sb = new StringBuilder();
int dato;
while ((dato = fis.read()) != -1) {
 sb.append((char) dato);
}
String contenido = sb.toString();
```

### CONTAR BYTES DE UN ARCHIVO

```
int contador = 0;
while (fis.read() != -1) contador++;
System.out.println("Bytes: " + contador);
```

### BUSCAR UN CARÁCTER CONCRETO

```
int dato, ocurrencias = 0;
while ((dato = fis.read()) != -1) {
 if (dato == 'a') ocurrencias++;
}
System.out.println("'a' aparece: " + ocurrencias);
```

### REEMPLAZAR CARÁCTER AL COPIAR

```
while ((dato = fis.read()) != -1) {
 fos.write(dato == ' ' ? '-' : dato); // espacio → guion
}
```

## 🔥 TABLA RESUMEN DEL BLOQUE

| Operación         | Clase                         | Método                           |
|-------------------|-------------------------------|----------------------------------|
| Leer archivo      | <code>FileInputStream</code>  | <code>read()</code> → int        |
| Escribir archivo  | <code>FileOutputStream</code> | <code>write(b)</code>            |
| Texto → bytes     | <code>String</code>           | <code>getBytes()</code>          |
| Byte → carácter   | –                             | <code>(char) dato</code>         |
| Detectar fin      | –                             | <code>read() == -1</code>        |
| Añadir al final   | <code>FileOutputStream</code> | constructor( <code>true</code> ) |
| Cierre automático | –                             | try-with-resources               |

**Regla de oro:** abre con try-with-resources · procesa en `while (read() != -1)` · captura `IOException`. Con esto el 95% de los casos están cubiertos.